

Laboratory Report: Lab 8

Zofia Warzycha, 280971

May 2026

1 Task Description

The goal of this lab was to extend the Space Station Management project with support for reading and saving data in CSV and text files. The main things to implement were:

- A CSV file `crew_members.csv` with crew member data and a static method `loadCrewMembersCSV` that reads it and returns an `ArrayList<CrewMember>`.
- Calculation and display of average crew age, average mission hours, and number of crew members per role, saved to `crew_report.txt`.
- A CSV file `supplies.csv` with supply data and a static method `loadSuppliesCSV` that reads it and returns an `ArrayList<Supply>`.
- Calculation of total cost per supply and grand total for all supplies, saved to `supplies_report.txt`.
- Both loading methods throw `IllegalArgumentException` if the file does not exist or contains incorrect data.
- An additional method in `Module` that saves all crew members assigned to a module to a separate CSV file named after the module.

2 Source Code

2.1 CSV Files

The `crew_members.csv` file stores one crew member per row. The first row is a header that is skipped during reading. Each data row has five fields separated by commas: name, age, role, specialization, and mission hours. The sample data in both CSV files was generated with the help of an AI chatbot (*claude.ai*, *free plan*), as it was suggested during the labs.

```
1 name,age,role,specialization,missionHours
2 Zofia Warzycha,35,ENGINEER,ship maintenance,1500.5
3 Jan Nowak,42,PILLOT,navigation,2100.0
4 Adam Kowalski,28,SCIENTIST,biology,800.0
5 Anna Wisniewska,30,MEDIC,emergency care,950.0
6 Piotr Zajac,50,COMMANDER,strategy,3000.0
7 Maria Lis,28,SCIENTIST,quantum mechanics,1200.0
8 Tomasz Brzoza,35,ENGINEER,electronics,600.0
9 Katarzyna Kwiatkowska,40,PILLOT,asteroid mapping,1800.0
```

Listing 1: `crew_members.csv`

The `supplies.csv` file follows the same pattern with three fields per row: name, quantity, and unit cost.

```
1 name,quantity,unitCost
2 Oxygen Tank,10,49.99
3 Medical Kit,5,199.00
4 Titanium Wrench,3,89.50
5 Freeze-dried Food,100,12.50
```

```

6 Water Purifier,2,349.99
7 Space Suit Patch,20,15.75
8 Battery Pack,15,74.90

```

Listing 2: supplies.csv

2.2 loadCrewMembersCSV Method

The `loadCrewMembersCSV` method opens the file with a `Scanner`, skips the header line, and then reads each row with `split(",")`. If the file does not exist, or a row does not have exactly five fields, or a number cannot be parsed, the method throws `IllegalArgumentException` with a message that includes the line number where the error occurred. The `Role` is parsed with `Role.valueOf()` after trimming and converting to uppercase, so the CSV values are case-insensitive.

```

1 public static ArrayList<CrewMember> loadCrewMembersCSV(String filename) {
2     File file = new File(filename);
3     if (!file.exists()) {
4         throw new IllegalArgumentException(
5             "File not found: " + filename);
6     }
7     ArrayList<CrewMember> result = new ArrayList<>();
8     try (Scanner scanner = new Scanner(file)) {
9         if (scanner.hasNextLine()) scanner.nextLine(); // skip header
10        int lineNumber = 1;
11        while (scanner.hasNextLine()) {
12            lineNumber++;
13            String line = scanner.nextLine().trim();
14            if (line.isEmpty()) continue;
15            String[] parts = line.split(",");
16            if (parts.length != 5) {
17                throw new IllegalArgumentException(
18                    "Invalid row at line " + lineNumber
19                    + ": expected 5 fields, got " + parts.length);
20            }
21            try {
22                String name = parts[0].trim();
23                int age = Integer.parseInt(parts[1].trim());
24                Role role = Role.valueOf(
25                    parts[2].trim().toUpperCase());
26                String specialization = parts[3].trim();
27                double missionHours =
28                    Double.parseDouble(parts[4].trim());
29                result.add(new CrewMember(name, age, role,
30                    specialization, missionHours));
31            } catch (NumberFormatException e) {
32                throw new IllegalArgumentException(
33                    "Number format error at line "
34                    + lineNumber + ": " + e.getMessage());
35            }
36        }
37    } catch (IOException e) {
38        throw new IllegalArgumentException(
39            "Could not read file: " + filename);
40    }
41    return result;
42 }

```

Listing 3: loadCrewMembersCSV in FileHelper.java

2.3 saveCrewReport Method

The `saveCrewReport` method writes the statistics to a plain text file using a `PrintWriter` wrapped around a `FileWriter`. Average age and average mission hours are calculated with `stream()` and `mapToInt` / `mapToDouble`. The count per role is built by iterating over all `Role` enum values and filtering the list.

```

1 public static void saveCrewReport(String filename,
2                                 ArrayList<CrewMember> crew) {
3     try (PrintWriter writer =
4         new PrintWriter(new FileWriter(filename))) {
5
6         writer.println("SPACE STATION CREW REPORT");
7         writer.println("Total crew members: " + crew.size());
8
9         double avgAge = crew.stream()
10            .mapToInt(CrewMember::getAge).average().orElse(0);
11         writer.printf("Average age: %.2f%n", avgAge);
12
13         double avgHours = crew.stream()
14            .mapToDouble(CrewMember::getMissionHours)
15            .average().orElse(0);
16         writer.printf("Average mission hours: %.2f%n", avgHours);
17
18         writer.println("\nCrew members per role:");
19         for (Role role : Role.values()) {
20             long count = crew.stream()
21                .filter(c -> c.getRole() == role).count();
22             if (count > 0) {
23                 writer.printf(" %-12s : %d%n", role, count);
24             }
25         }
26     } catch (IOException e) {
27         System.out.println(
28             "Error saving crew report: " + e.getMessage());
29     }
30 }

```

Listing 4: saveCrewReport in FileHelper.java

2.4 loadSuppliesCSV Method

The loadSuppliesCSV method works the same way as loadCrewMembersCSV but expects three fields per row. The unit cost is parsed as a `BigDecimal` to avoid floating-point rounding errors. The `isConsumable` flag defaults to `true` when loading from CSV, because the file format does not include that field.

```

1 public static ArrayList<Supply> loadSuppliesCSV(String filename) {
2     File file = new File(filename);
3     if (!file.exists()) {
4         throw new IllegalArgumentException(
5             "File not found: " + filename);
6     }
7     ArrayList<Supply> result = new ArrayList<>();
8     try (Scanner scanner = new Scanner(file)) {
9         if (scanner.hasNextLine()) scanner.nextLine(); // skip header
10        int lineNumber = 1;
11        while (scanner.hasNextLine()) {
12            lineNumber++;
13            String line = scanner.nextLine().trim();
14            if (line.isEmpty()) continue;
15            String[] parts = line.split(",");
16            if (parts.length != 3) {
17                throw new IllegalArgumentException(
18                    "Invalid row at line " + lineNumber
19                    + ": expected 3 fields, got " + parts.length);
20            }
21            try {
22                String name = parts[0].trim();
23                int quantity = Integer.parseInt(parts[1].trim());
24                BigDecimal cost = new BigDecimal(parts[2].trim());
25                result.add(new Supply(name, quantity, cost, true));
26            } catch (NumberFormatException e) {
27                throw new IllegalArgumentException(
28                    "Number format error at line "

```

```

29         + lineNumber + ": " + e.getMessage());
30     }
31 }
32 } catch (IOException e) {
33     throw new IllegalArgumentException(
34         "Could not read file: " + filename);
35 }
36 return result;
37 }

```

Listing 5: loadSuppliesCSV in FileHelper.java

2.5 saveSuppliesReport Method

The `saveSuppliesReport` method formats each supply as a table row with the name, quantity, unit cost, and total cost. The grand total is accumulated with `BigDecimal.add()` to keep the arithmetic exact.

```

1 public static void saveSuppliesReport(String filename,
2                                     ArrayList<Supply> supplies) {
3     try (PrintWriter writer =
4         new PrintWriter(new FileWriter(filename))) {
5
6         writer.println("SPACE STATION SUPPLIES REPORT");
7         writer.printf("%-22s | %5s | %10s | %12s%n",
8             "Name", "Qty", "Unit cost", "Total cost");
9         writer.println(
10             "-----");
11
12         BigDecimal grandTotal = BigDecimal.ZERO;
13         for (Supply s : supplies) {
14             BigDecimal total = s.getTotalPrice();
15             grandTotal = grandTotal.add(total);
16             writer.printf("%-22s | %5d | %10.2f | %12.2f%n",
17                 s.getName(), s.getQuantity(), s.getCost(), total);
18         }
19
20         writer.println(
21             "-----");
22         writer.printf("%-22s   %5s   %10s   %12.2f%n",
23             "TOTAL", "", "", grandTotal);
24
25     } catch (IOException e) {
26         System.out.println(
27             "Error saving supplies report: " + e.getMessage());
28     }
29 }

```

Listing 6: saveSuppliesReport in FileHelper.java

2.6 Module – saveCrewToCSV (Additional Task)

The additional task required saving all crew members of a selected module to a CSV file. I added a `name` field to `Module` and a second constructor that accepts the name. The `saveCrewToCSV` method derives the filename from the module name by converting it to lowercase and replacing spaces with underscores, so a module named `Science Lab` automatically saves to `science_lab_crew.csv`. The method iterates over the internal array and skips null slots.

```

1 public class Module {
2
3     private String name;
4     private CrewMember[] crewMembers;
5     private int maxCapacity;
6
7     public Module(String name, int maxCapacity) {
8         if (maxCapacity <= 0) {

```

```

9         throw new IllegalArgumentException(
10             "Module capacity must be greater than zero. Got: "
11             + maxCapacity);
12     }
13     this.name = name;
14     this.maxCapacity = maxCapacity;
15     this.crewMembers = new CrewMember[maxCapacity];
16 }
17
18 public void saveCrewToCSV() {
19     String filename = name.toLowerCase()
20         .replace(" ", "_") + "_crew.csv";
21     try (PrintWriter writer =
22         new PrintWriter(new FileWriter(filename))) {
23         writer.println(
24             "id,name,role,specialization,missionHours");
25         for (CrewMember c : crewMembers) {
26             if (c != null) {
27                 writer.printf("%s,%s,%s,%s,%.1f%n",
28                     c.getId(), c.getName(), c.getRole(),
29                     c.getSpecialization(), c.getMissionHours());
30             }
31         }
32         System.out.println(
33             "Module crew saved to: " + filename);
34     } catch (IOException e) {
35         System.out.println(
36             "Error saving module crew: " + e.getMessage());
37     }
38 }
39 }

```

Listing 7: saveCrewToCSV in Module.java

2.7 Tester Class

The tester calls all four methods in order, prints results to the console, and at the end tests the error handling by trying to load two files that do not exist.

```

1 public class List8_tester {
2
3     public static void main(String[] args) {
4
5         // ----- Crew members -----
6         System.out.println("1. CREW MEMBERS");
7         ArrayList<CrewMember> crew = null;
8         try {
9             crew = FileHelper.loadCrewMembersCSV(
10                 "crew_members.csv");
11         } catch (IllegalArgumentException e) {
12             System.out.println("[ERROR] " + e.getMessage());
13             return;
14         }
15
16         System.out.println("\n-- Loaded crew profiles --");
17         for (CrewMember c : crew) {
18             c.printProfile();
19         }
20
21         double avgAge = crew.stream()
22             .mapToInt(CrewMember::getAge).average().orElse(0);
23         double avgHours = crew.stream()
24             .mapToDouble(CrewMember::getMissionHours)
25             .average().orElse(0);
26         System.out.printf("Average age: %.2f%n", avgAge);
27         System.out.printf("Average mission hours: %.2f%n", avgHours);
28

```

```

29     System.out.println("\nCrew members per role:");
30     Map<Role, Long> perRole = new TreeMap<>();
31     for (CrewMember c : crew) {
32         perRole.merge(c.getRole(), 1L, Long::sum);
33     }
34     for (Map.Entry<Role, Long> entry : perRole.entrySet()) {
35         System.out.printf("  %-12s : %d\n",
36             entry.getKey(), entry.getValue());
37     }
38
39     FileHelper.saveCrewReport("crew_report.txt", crew);
40     System.out.println("Crew report saved to: crew_report.txt");
41
42     // ----- Supplies -----
43     System.out.println("\n2. SUPPLIES");
44     ArrayList<Supply> supplies = null;
45     try {
46         supplies = FileHelper.loadSuppliesCSV("supplies.csv");
47     } catch (IllegalArgumentException e) {
48         System.out.println("[ERROR] " + e.getMessage());
49         return;
50     }
51
52     System.out.println("\n-- Loaded supplies --");
53     BigDecimal grandTotal = BigDecimal.ZERO;
54     for (Supply s : supplies) {
55         BigDecimal total = s.getTotalPrice();
56         grandTotal = grandTotal.add(total);
57         System.out.printf(
58             "  %-22s | qty: %3d | unit: %8.2f"
59             + " | total: %10.2f\n",
60             s.getName(), s.getQuantity(),
61             s.getCost(), total);
62     }
63     System.out.printf(
64         "Total cost of all supplies: %.2f\n", grandTotal);
65     FileHelper.saveSuppliesReport(
66         "supplies_report.txt", supplies);
67     System.out.println(
68         "Supplies report saved to: supplies_report.txt");
69
70     // ----- Module crew export -----
71     System.out.println("\n3. MODULE CREW EXPORT");
72     Module scienceLab = new Module("Science Lab", 4);
73     try {
74         scienceLab.addCrewMember(crew.get(2));
75         scienceLab.addCrewMember(crew.get(5));
76         scienceLab.addCrewMember(crew.get(3));
77     } catch (ModuleFullException e) {
78         System.out.println("[ERROR] " + e.getMessage());
79     }
80     System.out.println("Module \"" + scienceLab.getName()
81         + "\" has " + scienceLab.getCurrentSize()
82         + "/" + scienceLab.getMaxCapacity()
83         + " crew members.");
84     scienceLab.saveCrewToCSV();
85
86     // ----- Error handling -----
87     System.out.println("\n4. ERROR HANDLING");
88     try {
89         FileHelper.loadCrewMembersCSV("missing_file.csv");
90     } catch (IllegalArgumentException e) {
91         System.out.println(
92             "[OK] File missing: " + e.getMessage());
93     }
94     try {
95         FileHelper.loadSuppliesCSV("bad_file.csv");
96     } catch (IllegalArgumentException e) {

```

```

97         System.out.println(
98             "[OK] File missing: " + e.getMessage());
99     }
100
101     System.out.println("\nALL TASKS COMPLETED");
102 }
103 }

```

Listing 8: Key sections from List8_tester.java

3 Console Output

Example output produced by the List8_tester program:

```

"C:\Program Files\Java\jdk-25\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.2.3\lib\idea_rt.jar=65085" -Dfile.encoding=UTF-8 -D
1. CREW MEMBERS

-- Loaded crew profiles --
Hello! Meet Zofia Warzycha, a 35-year-old ENGINEER. This crew member specializes in ship maintenance and has already completed 1500.5 mission hours.
Hello! Meet Jan Nowak, a 42-year-old PILOT. This crew member specializes in navigation and has already completed 2100.0 mission hours.
Hello! Meet Adam Kowalski, a 28-year-old SCIENTIST. This crew member specializes in biology and has already completed 800.0 mission hours.
Hello! Meet Anna Wisniewska, a 30-year-old MEDIC. This crew member specializes in emergency care and has already completed 950.0 mission hours.
Hello! Meet Piotr Zajac, a 50-year-old COMMANDER. This crew member specializes in strategy and has already completed 3000.0 mission hours.
Hello! Meet Maria Lis, a 28-year-old SCIENTIST. This crew member specializes in quantum mechanics and has already completed 1200.0 mission hours.
Hello! Meet Tomasz Brzoza, a 35-year-old ENGINEER. This crew member specializes in electronics and has already completed 600.0 mission hours.
Hello! Meet Katarzyna Kwiatkowska, a 40-year-old PILOT. This crew member specializes in asteroid mapping and has already completed 1800.0 mission hours.

Average age: 36,00
Average mission hours: 1493,81

Crew members per role:
COMMANDER : 1
PILOT      : 2
ENGINEER   : 2
SCIENTIST  : 2
MEDIC      : 1

Crew report saved to: crew_report.txt

2. SUPPLIES

-- Loaded supplies --
Oxygen Tank      | qty: 10 | unit: 49,99 | total: 499,90
Medical Kit      | qty: 5  | unit: 199,00 | total: 995,00
Titanium Wrench  | qty: 3  | unit: 89,50 | total: 268,50
Freeze-dried Food | qty: 100 | unit: 12,50 | total: 1250,00
Water Purifier   | qty: 2  | unit: 349,99 | total: 699,98
Space Suit Patch | qty: 20 | unit: 15,75 | total: 315,00
Battery Pack     | qty: 15 | unit: 74,90 | total: 1123,50

Total cost of all supplies: 5151,88
Supplies report saved to: supplies_report.txt

```

Figure 1: Console output – crew loading, statistics, and supply costs

```
3. MODULE CREW EXPORT
Adam Kowalski added to module "Science Lab".
Maria Lis added to module "Science Lab".
Anna Wisniewska added to module "Science Lab".
Module "Science Lab" has 3/4 crew members.
Module crew saved to: science_lab_crew.csv

4. ERROR HANDLING
[OK] File missing: File not found: missing_file.csv
[OK] File missing: File not found: bad_file.csv

ALL TASKS COMPLETED

Process finished with exit code 0
```

Figure 2: Console output – module crew export and error handling

4 Conclusions

- I learned how to read a CSV file line by line using `Scanner` and `split(",")`. The most important thing is to validate every row before creating an object from it – checking the number of fields and catching `NumberFormatException` prevents the program from crashing on bad input.
- Throwing `IllegalArgumentException` from the loading methods instead of returning an empty list is the right design choice because it forces the caller to notice that something went wrong. A silent empty list could easily cause a bug later in the program.
- I used `PrintWriter` with `FileWriter` to write the reports. Wrapping them in a try-with-resources block guarantees that the file is closed even if an exception is thrown, which prevents data loss.
- Deriving the output filename from the module name (`name.toLowerCase().replace(" ", "_")`) means the method does not need any extra parameter. The caller just calls `saveCrewToCSV()` and the correct file is created automatically.
- I noticed that the CSV files have to be placed in the project root folder (the working directory of the JVM) for `new File("...")` to find them without an absolute path.